

=====

=====

GUI FLUENT API ДЛЯ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА (ФИНАЛЬНАЯ ВЕРСИЯ 1.1)

Проект: Mozart ERP

=====

=====

ОГЛАВЛЕНИЕ

1. Общая концепция GUI
2. ERPMozart.gui — точка входа
3. Forms — управление формами (немодальные, с родителем)
4. Form — конкретная форма
5. Controls — фабрика контролов
6. DataBinding — привязка данных (manual по умолчанию)
7. События и взаимодействие форм
8. Grid — таблицы
9. Tree — деревья
10. Полные примеры использования

=====

=====

1. ОБЩАЯ КОНЦЕПЦИЯ GUI

=====

=====

GUI построен на Fluent API для данных.

Все формы немодальные. Каждая знает, кто её открыл (родитель).
При закрытии дочерняя форма может вернуть результат родителю.

Изменения в контролах — только в локальной модели.
Сохранение в БД — только по явной команде (кнопка "Сохранить" / "ОК").
Это защищает от лавины версий и даёт пользователю контроль.

Для версионизируемых сущностей: при сохранении старая версия закрывается,
создаётся новая. При отмене — ничего не меняется.

=====

=====

2. ERPMozart.gui — ТОЧКА ВХОДА В GUI

=====

=====

ERPMozart.gui:

Точка входа для GUI. Содержит всё, что связано с интерфейсом.

Свойства:

main_window:	MainWindow	— главное окно
forms:	Forms	— управление формами
controls:	Controls	— фабрика контролов

bindings: DataBinding — привязка данных
events: Events — система событий
theme: Theme — тема
translator: Translator — переводчик

Методы:

gui.run() — запустить приложение
gui.show_message(text, type) — info / warning / error
gui.confirm(text) -> bool — диалог Да/Нет
gui.open_file_dialog() -> str | None
gui.save_file_dialog() -> str | None

class MainWindow:

"""Главное окно приложения."""

def set_title(self, title: str): ...
def set_size(self, w: int, h: int): ...
def close(self): ...

3. Forms — УПРАВЛЕНИЕ ФОРМАМИ

class Forms:

"""

Все формы немодальные. Каждая имеет уникальный идентификатор.
При повторном открытии — активируется существующая.

"""

def open(self,
 identifier: str | int,
 parent: 'Form' = None,
 **params) -> 'Form':
 """

Открыть форму.

identifier: str — 'entity_editor', 'person_list'
 int — meta.forms.id

parent: Form — кто открыл (для возврата результата)
params: dict — контекст (entity_id=2, person_id=123)

Если форма уже открыта — активируется, params игнорируются.
"""

def close(self, identifier: str | int): ...

def get(self, identifier: str | int) -> 'Form | None': ...

```
def list_open(self) -> list['Form']: ...
```

```
class Form:
```

```
    """
```

```
    Немодальная форма. Знает родителя.
```

```
    Содержит контролы, вкладки, тулбар.
```

```
    """
```

```
    # Идентификация
```

```
    identifier: str          # уникальный ID
```

```
    title: str
```

```
    # Иерархия
```

```
    parent: 'Form | None'    # кто открыл (null для главных форм)
```

```
    data_context: dict       # параметры, переданные при открытии
```

```
    # Состояние
```

```
    is_modified: bool        # были ли изменения в контролах
```

```
    is_closing: bool         # в процессе закрытия
```

```
    # ===== ЖИЗНЕННЫЙ ЦИКЛ =====
```

```
    def on_open(self, **params):
```

```
        """
```

```
        Вызывается при открытии. Здесь создаются контролы.
```

```
        params — то, что передали в Forms.open().
```

```
        """
```

```
    def on_close(self) -> bool:
```

```
        """
```

```
        Вызывается при закрытии.
```

```
        Вернуть False чтобы запретить закрытие.
```

```
        По умолчанию: если is_modified — спросить "Сохранить?".
```

```
        """
```

```
    def on_activate(self):
```

```
        """Форму принесли на передний план."""
```

```
    def on_parent_closed(self, parent_form: 'Form'):
```

```
        """Родительскую форму закрыли."""
```

```
    # ===== КОНТРОЛЫ =====
```

```
    def add_control(self, control: 'Control', area: str = 'center'):
```

```
        """
```

```
        Добавить контрол. area:
```

```
        'top', 'bottom', 'left', 'right', 'center'
```

```
        'tab:ИмяВкладки'
```

```
        """
```

```
def add_tab(self, name: str) -> 'TabPage':
    """Добавить вкладку."""
```

```
def add_toolbar(self, buttons: list[dict]):
    """
    Кнопки тулбара:
    [
        {'text': 'Сохранить', 'action': self.save, 'icon': 'save'},
        {'text': 'Закрыть', 'action': self.close, 'icon': 'close'},
    ]
    """
```

```
def get_control(self, name: str) -> 'Control | None': ...
```

```
# ===== ДЕЙСТВИЯ =====
```

```
def save(self):
    """Сохранить все изменения в БД."""
    gui.bindings.save()
```

```
def rollback(self):
    """Откатить изменения (перечитать из БД)."""
    gui.bindings.rollback()
```

```
def close(self):
    """Закрыть форму (с проверкой is_modified)."""
```

```
def return_result(self, result: dict):
    """
    Вернуть результат родителю.
    У родителя вызовется on_child_result(result).
    """
```

```
# ===== ОБРАТНЫЙ ВЫЗОВ =====
```

```
def on_child_result(self, child_form: 'Form', result: dict):
    """
    Вызывается когда дочерняя форма закрывается с результатом.
    По умолчанию ничего не делает. Переопределите.
    """
```

```
=====
=====
4. Controls — ФАБРИКА КОНТРОЛОВ
=====
=====
```

```
class Controls:
    """Создание элементов управления."""
```

```
def textbox(self, name: str, label: str = None, **props) -> 'TextBox': ...
```

```

def numberbox(self, name: str, label: str = None, **props) -> 'NumberBox': ...
def datebox(self, name: str, label: str = None, **props) -> 'DateBox': ...
def checkbox(self, name: str, label: str = None, **props) -> 'CheckBox': ...
def combobox(self, name: str, label: str = None, **props) -> 'ComboBox':
    """props: items=[{'value': 1, 'text': 'Да'}, ...]"""
def memo(self, name: str, label: str = None, **props) -> 'Memo': ...
def button(self, name: str, text: str, action: callable = None) -> 'Button': ...
def label(self, text: str, **props) -> 'Label': ...
def grid(self, name: str, **props) -> 'Grid': ...
def tree(self, name: str, **props) -> 'Tree': ...
def file_picker(self, name: str, label: str = None, **props) -> 'FilePicker':
    """Для полей типа G (bytea + _name)."""
def reference(self, name: str, label: str = None, **props) -> 'Reference':
    """Для полей типа R (ссылка на другую сущность)."""

```

```

class Control:

```

```

    """Базовый контрол."""

```

```

    name: str
    label: str
    value: Any          # текущее значение (локальное)
    original_value: Any # исходное значение из БД
    is_readonly: bool
    is_required: bool
    is_visible: bool
    is_enabled: bool

```

```

    def set_value(self, value: Any):
        """Установить значение (локально)."""

```

```

    def get_value(self) -> Any: ...

```

```

    def reset(self):
        """Вернуть исходное значение."""

```

```

    def is_dirty(self) -> bool:
        """Изменилось ли значение по сравнению с original_value."""

```

```

    def on_change(self, callback: callable):
        """callback(new_value) — при изменении."""

```

```

class FilePicker(Control):

```

```

    """Выбор файла. Для типа G."""
    def load_file(self, path: str): ...
    def get_data(self) -> bytes: ...
    def get_file_name(self) -> str: ...

```

```

class Reference(Control):

```

```

    """Ссылка на другую сущность. Для типа R."""
    def set_entity_filter(self, entity_alias: str): ...

```

```
def open_selector(self): ...
```

```
=====
5. DataBinding — ПРИВЯЗКА ДАННЫХ (РУЧНОЕ СОХРАНЕНИЕ)
=====
```

```
class DataBinding:
```

```
    """
```

```
    Привязка контролов к данным.
```

```
    ВАЖНО: Изменения в контролах — только в локальной модели.
```

```
    Сохранение в БД — только через binding.save() (по кнопке "Сохранить").
```

```
    """
```

```
def bind(self,
```

```
    control: Control,
```

```
    entity_path: str,
```

```
    field_name: str):
```

```
    """
```

```
    Привязать контрол к полю.
```

```
    entity_path — Fluent-путь:
```

```
    'entity("person").item(123).version()'
```

```
    'entity("person").item(123).version().daughter_table("FILES").item(456).version()'
```

```
    При вызове bind():
```

```
    1. Загружает текущее значение из БД
```

```
    2. Устанавливает control.value и control.original_value
```

```
    3. Подписывается на control.on_change() для отслеживания is_dirty
```

```
    Пример:
```

```
    binding.bind(txt_lastname, 'entity("person").item(123).version()', 'clastname')
```

```
    """
```

```
def bind_grid(self, grid: 'Grid', entity_alias: str,
```

```
    where: str = None, params: dict = None,
```

```
    order_by: str = None, limit: int = 100):
```

```
    """
```

```
    Привязать таблицу к списку экземпляров.
```

```
    Загружает данные из БД сразу.
```

```
    """
```

```
def bind_tree(self, tree: 'Tree', entity_alias: str,
```

```
    parent_field: str = 'parentid'):
```

```
    """Привязать дерево к иерархической сущности."""
```

```
def save(self):
```

```
    """
```

```
    Сохранить ВСЕ грязные (is_dirty) контролы в БД.
```

Для версионизируемых сущностей:

1. Если были изменения полей ext-таблицы:
 - а) Старая версия: tdt_end=now, cversionstatus='History'
 - б) Новая версия: INSERT entity_versions + INSERT/UPDATE ext_*
2. Если изменялись только неверсионизируемые поля — UPDATE.

После сохранения:

- Все original_value обновляются
 - is_dirty сбрасывается
 - is_modified формы = false
- """

def rollback(self):

"""

Откатить все изменения.

Для каждого контрола: value = original_value (перечитать из БД).

Никаких запросов к БД (кроме несохранённых новых экземпляров).

"""

def is_modified(self) -> bool:

"""Есть ли хотя бы один грязный контрол."""

def refresh(self):

"""Перечитать все привязанные контролы из БД.

original_value обновляется, is_dirty сбрасывается."""

=====

6. СОБЫТИЯ И ВЗАИМОДЕЙСТВИЕ ФОРМ

=====

class Events:

"""

Система событий. Форма может подписаться на событие другой формы.

"""

def subscribe(self,

subscriber: Form,

event_name: str,

publisher_form_id: str,

callback: callable):

"""

event_name:

'item_selected' — выбор в таблице/дереве

'data_saved' — данные сохранены

'form_closed' — форма закрыта

callback(data: dict) — вызывается при событии.

"""

```
def emit(self, event_name: str, data: dict = None):
    """Отправить событие всем подписчикам."""
```

```
class Form:
```

```
    # ... продолжение ...
```

```
def on_close(self) -> bool:
    """По умолчанию: если is_modified — спросить."""
    if self.is_modified:
        answer = gui.confirm("Сохранить изменения?")
        if answer is None:    # Отмена
            return False
        elif answer:        # Да
            self.save()
    return True
```

```
@property
def is_modified(self) -> bool:
    """Есть ли изменения в любом контроле формы."""
    return gui.bindings.is_modified()
```

```
=====
=====
7. Grid — ТАБЛИЦЫ
=====
=====
```

```
class Grid(Control):
```

```
    """Таблица."""
```

```
def set_columns(self, columns: list[dict]):
    """
    columns = [
        {'name': 'cname', 'header': 'ФИО', 'width': 300},
        {'name': 'clastname', 'header': 'Фамилия', 'width': 150},
    ]
    """
```

```
def set_data(self, rows: list[dict]): ...
def get_selected(self) -> dict | None: ...
```

```
def on_select(self, callback: callable):
    """callback(selected_row) — при выборе строки."""
```

```
def on_double_click(self, callback: callable):
    """callback(selected_row) — при двойном клике."""
```



```
=====
=====
8. Tree — ДЕРЕВЬЯ
=====
=====
```

```
class Tree(Control):
    """Дерево."""

    def set_data(self, nodes: list[dict]):
        """
        nodes = [
            {'id': 1, 'parent_id': None, 'data': {'cname': 'Корень'}, 'children': [...]}
        ]
        """

    def get_selected(self) -> dict | None: ...
    def expand_all(self): ...

    def on_select(self, callback: callable): ...
    def on_expand(self, callback: callable): ...
```

```
=====
=====
9. ПОЛНЫЕ ПРИМЕРЫ
=====
=====
```

```
# ===== 1. ПРОСТАЯ ФОРМА РЕДАКТИРОВАНИЯ =====
```

```
class PersonEditForm(Form):
    def on_open(self, person_id: int, **params):
        self.set_title("Редактирование персоны")

        # Контролы
        self.add_control(gui.controls.textbox('clastname', 'Фамилия'))
        self.add_control(gui.controls.textbox('cfirstname', 'Имя'))
        self.add_control(gui.controls.textbox('csecondname', 'Отчество'))
        self.add_control(gui.controls.datebox('dbirthdate', 'Дата рождения'))
        self.add_control(gui.controls.textbox('cinn', 'ИНН'))

        # Привязка
        path = f'entity("person").item({person_id}).version()'
        for ctl in [self.get_control(n) for n in ['clastname', 'cfirstname', 'csecondname', 'dbirthdate',
        'cinn']]:
            gui.bindings.bind(ctl, path, ctl.name)

        # Тулбар
        self.add_toolbar([
            {'text': 'Сохранить', 'action': self.save},
            {'text': 'Отмена', 'action': self.rollback},
```

```
        {'text': 'Закреть', 'action': self.close},
    )
```

===== 2. ФОРМА СО СПИСКОМ =====

```
class PersonListForm(Form):
    def on_open(self, **params):
        self.set_title("Персоны")

        # Таблица
        grid = gui.controls.grid('persons')
        grid.set_columns([
            {'name': 'cname', 'header': 'ФИО', 'width': 300},
            {'name': 'clastname', 'header': 'Фамилия', 'width': 150},
            {'name': 'cinn', 'header': 'ИНН', 'width': 120},
        ])
        self.add_control(grid)

        # Загрузка данных
        gui.bindings.bind_grid(grid, 'person',
            where="cversionstatus = 'Active'",
            order_by="clastname ASC"
        )

        # Двойной клик — редактирование
        def edit_person(row):
            gui.forms.open('person_edit', parent=self, person_id=row['instance_id'])
        grid.on_double_click(edit_person)

        # Тулбар
        self.add_toolbar([
            {'text': 'Добавить', 'action': self.add_person},
            {'text': 'Обновить', 'action': lambda: gui.bindings.refresh()},
        ])

    def add_person(self):
        # Создаём нового
        item = mozart.data.entity('person').create_item(cname='Новая персона')
        # Открываем на редактирование
        gui.forms.open('person_edit', parent=self, person_id=item.instance_id)
```

===== 3. ФОРМА С ВКЛАДКАМИ =====

```
class PersonFullForm(Form):
    def on_open(self, person_id: int, **params):
        self.set_title("Карточка персоны")

        # Вкладка "Основное"
        tab_main = self.add_tab("Основное")
        tab_main.add_control(gui.controls.textbox('clastname', 'Фамилия'))
        tab_main.add_control(gui.controls.textbox('cfirstname', 'Имя'))
```

```

# Вкладка "Файлы"
tab_files = self.add_tab("Файлы")
grid = gui.controls.grid('files')
grid.set_columns([
    {'name': 'file_data_name', 'header': 'Файл', 'width': 250},
    {'name': 'mime_type', 'header': 'Тип', 'width': 150},
])
tab_files.add_control(grid)

# Привязка основных полей
path = f'entity("person").item({person_id}).version()'
gui.bindings.bind(self.get_control('clastname'), path, 'clastname')
gui.bindings.bind(self.get_control('cfirstname'), path, 'cfirstname')

# Данные файлов
file_path = f'entity("person").item({person_id}).version().daughter_table("FILES")'
# Загружаем список файлов
files = mozart.data.entity('person').item(person_id).version()\
    .daughter_table('FILES').items()
grid.set_data(files)

```

===== 4. ВОЗВРАТ РЕЗУЛЬТАТА РОДИТЕЛЮ =====

```

class SelectPersonForm(Form):
    """Форма выбора персоны (возвращает результат)."""

    def on_open(self, **params):
        grid = gui.controls.grid('persons')
        grid.set_columns([...])
        self.add_control(grid)
        gui.bindings.bind_grid(grid, 'person', where="cversionstatus = 'Active'")

        self.add_toolbar([
            {'text': 'Выбрать', 'action': self.select},
            {'text': 'Отмена', 'action': self.close},
        ])

    def select(self):
        row = self.get_control('persons').get_selected()
        if row:
            self.return_result({
                'action': 'selected',
                'person_id': row['instance_id'],
                'person_name': row['cname']
            })
            self.close()

# Вызов из родительской формы:
class MainForm(Form):
    def open_selector(self):
        gui.forms.open('select_person', parent=self)

```

```
def on_child_result(self, child_form: Form, result: dict):
    if result['action'] == 'selected':
        self.get_control('person_ref').set_value(
            result['person_id'], result['person_name']
        )
```

```
=====
=====
КОНЕЦ ОПИСАНИЯ
=====
=====
```